

OBJECT ORIENTED PROGRAMMING (CDT2001)

**B.TECH. SEM II
UNIT V**

- Generics
- Generics class with two type parameter
- Bounded Generics
- Collection Classes
- ArrayList
- Linked List
- Hashset
- TreeSet

- Java Generics were introduced in JDK 5.0 with the aim of reducing bugs and adding an extra layer of abstraction over types.
- The Java Generics allows us to create a single class, interface, and method that can be used with different types of data (objects).
- This helps us to reuse our code.
- Generics add stability to your code by making more of your bugs detectable at compile time.”

It would be nice if we could write a single sort method that could sort the elements in an Integer array, a String array, or an array of any type that supports ordering.

Java Generic methods and generic classes enable programmers to specify, with a single method declaration, a set of related methods, or with a single class declaration, a set of related types, respectively.

Generics also provide compile-time type safety that allows programmers to catch invalid types at compile time.

Using Java Generic concept, we might write a generic method for sorting an array of objects, then invoke the generic method with Integer arrays, Double arrays, String arrays and so on, to sort the array elements..

- You can write a single generic method declaration that can be called with arguments of different types. Based on the types of the arguments passed to the generic method, the compiler handles each method call appropriately.
- we can also create a method that can be used with any type of data. Such a class is known as Generics Method.

- Following are the rules to define Generic Methods –
- All generic method declarations have a type parameter section delimited by angle brackets (< and >) that precedes the method's return type.
- Each type parameter section contains one or more type parameters separated by commas. A type parameter, also known as a type variable, is an identifier that specifies a generic type name.
- A generic method's body is declared like that of any other method. Note that type parameters can represent only reference types, not primitive types (like int, double and char).

- We can create a class that can be used with any type of data. Such a class is known as Generics Class.
- We can define our own classes with generics type. A generic type is a class or interface that is parameterized over types. We use angle brackets (<>) to specify the type parameter.

Example

```
public class GenericsType<T> {  
    private T t;  
  
    public T get(){  
        return this.t;  
    }  
  
    public void set(T t1){  
        this.t=t1;  
    }  
  
    public static void main(String args[]){  
        GenericsType<String> type = new GenericsType<>();  
        type.set("Pankaj"); //valid  
  
        GenericsType type1 = new GenericsType(); //raw  
        type1.set("Pankaj"); //valid  
        type1.set(10); //valid and autoboxing support  
    }  
}
```


- Generics was added in Java 5 to provide compile-time type checking and removing risk of ClassCastException that was common while working with collection classes.
- The whole collection framework was re-written to use generics for type-safety.

```
List list = new ArrayList();  
list.add("abc");  
list.add(new Integer(5)); //OK  
  
for(Object obj : list){  
    //type casting leading to ClassCastException at runtime  
    String str=(String) obj;  
}
```

```
class NonGen{
    Object ob;
    NonGen(Object o)
    {      ob=o;    }
    Object ReturnObject()
    {      return ob;    }
    void ShowType()
    {      ob.getClass().getName();
    }
}
```

```
public class GenDemo {
    public static void main(String ar[])
    {
        NonGen IntOb=new NonGen(88);
        int i=(int) IntOb.ReturnObject();
        System.out.println("i=" + i);
        NonGen StrOb=new NonGen("Hello");
        String s=(String) StrOb.ReturnObject();
        System.out.println("s=" + s);
        IntOb=StrOb;
        i=(int) IntOb.ReturnObject();
        System.out.println("i=" + i);
    } }
```

```
class NonGen<T>{  
    T ob;  
    NonGen(T o)  
    {        ob=o;    }  
    T ReturnObject()  
    {        return ob;    }  
    void ShowType()  
    {        ob.getClass().getName();  
    }  
}
```

```
public class GenDemo {  
    public static void main(String ar[])  
    {  
        NonGen<Integer> IntOb=new  
NonGen<>(88);  
        int i= IntOb.ReturnObject();  
        System.out.println("i=" + i);  
        NonGen<String> StrOb=new  
NonGen<>("Hello");  
        String s= StrOb.ReturnObject();  
        System.out.println("s=" + s);  
        IntOb=StrOb;  
        i= IntOb.ReturnObject();  
        System.out.println("i=" + i);  
    } }
```

Type Parameter Naming Conventions

- Java Generic Type Naming convention helps us understanding code easily and having a naming convention is one of the best practices of Java programming language.
- So generics also comes with its own naming conventions. Usually, type parameter names are single, uppercase letters to make it easily distinguishable from java variables.
- The most commonly used type parameter names are:
 - E - Element (used extensively by the Java Collections Framework)
 - K - Key
 - N - Number
 - T - Type
 - V - Value
 - S,U,V etc. - 2nd, 3rd, 4th types

A Generic Class with Two Type Parameters

```
class TwoGen<T, V>
{
    T ob1;
    V ob2;
    TwoGen(T o1, V o2) {
        ob1 = o1;
        ob2 = o2; }
    void showTypes() {
        System.out.println("Type of T is " + ob1.getClass().getName());
        System.out.println("Type of V is " +
            ob2.getClass().getName()); }
    T getob1() { return ob1; }
    V getob2() { return ob2; }
}
```

```
class SimpGen {  
    public static void main(String args[]) {  
        TwoGen<Integer, String> tgObj = new TwoGen<Integer, String>(88, "Generics");  
        // Show the types.  
        tgObj.showTypes();  
        // Obtain and show values.  
        int v = tgObj.getob1();  
        System.out.println("value: " + v);  
        String str = tgObj.getob2();  
        System.out.println("value: " + str);  
    }  
}
```

- Sometimes we don't want the whole class to be parameterized, in that case, we can create java generics method.
- Since the constructor is a special kind of method, we can use generics type in constructors too.

Example

```
public class GenericMethodTest
{
    public static < E > void printArray( E[] inputArray
)
    {
        for ( E element : inputArray )
        {
            System.out.printf( "%s ", element );
        }
        System.out.println();
    }
}
```

```
public static void main( String args[] )
{
    Integer[] intArray = { 1, 2, 3, 4, 5 };
    Double[] doubleArray = { 1.1, 2.2, 3.3, 4.4 };
    Character[] charArray = { 'H', 'E', 'L', 'L', 'O' };

    System.out.println( "Array integerArray
contains:" );
    printArray( intArray ); // pass an Integer array

    System.out.println( "\nArray doubleArray
contains:" );
    printArray( doubleArray ); // pass a Double
array

    System.out.println( "\nArray characterArray
contains:" );
    printArray( charArray ); // pass a Character
array
} }
```



```
public class GenericsMethods {  
    //Java Generic Method  
    public static <T> boolean isEqual(GenericsType<T> g1,  
    GenericsType<T> g2){  
        return g1.get().equals(g2.get());  
    }  
  
    public static void main(String args[]){  
        GenericsType<String> g1 = new GenericsType<>();  
        g1.set("Pankaj");  
  
        GenericsType<String> g2 = new GenericsType<>();  
        g2.set("Pankaj");  
  
        boolean isEqual = GenericsMethods.  
<String>isEqual(g1, g2);  
        //above statement can be written simply as  
        isEqual = GenericsMethods.isEqual(g1, g2);  
        //This feature, known as type inference, allows  
you to invoke a generic method as an ordinary method, without  
specifying a type between angle brackets.  
        //Compiler will infer the type that is needed  
    }  
}
```

- type parameters can be bounded.
- Bounded means “restricted”, we can restrict types that can be accepted by a method.
- For example, we can specify that a method accepts a type and all its subclasses (upper bound) or a type all its superclasses (lower bound).

- To declare an upper bounded type we use the keyword `extends` after the type followed by the upper bound that we want to use.
- For example:

```
public <T extends Number> List<T> fromArrayToList(T[] a) {  
    ...  
}
```

- There may be times when you want to restrict the types that can be used as type arguments in a parameterized type.
- For example, a method that operates on numbers might only want to accept instances of Number or its subclasses. This is what bounded type parameters are for.
- To declare a bounded type parameter, list the type parameter's name, followed by the extends keyword, followed by its upper bound, which in this example is Number.
- Note: "extends" (as in classes) or "implements" (as in interfaces).

```
class Stats<T extends Number>{
    T[] nums;
    Stats(T[] o) {
        nums = o;}

    double average() {
        double sum = 0.0;
        for(int i=0; i < nums.length; i++)
            sum += nums[i].doubleValue();
        return sum / nums.length;}
}
```

```
class BoundsDemo {
    public static void main(String args[]) {
        Integer inums[] = { 1, 2, 3, 4, 5 };
        Stats<Integer> iob = new
            Stats<Integer>(inums);
        double v = iob.average();
        System.out.println("iob average is " + v);
        Double dnums[] = { 1.1, 2.2, 3.3, 4.4, 5.5 };
        Stats<Double> dob = new
            stats<Double>(dnums);
        double w = dob.average();
        System.out.println("dob average is " + w);
        // String strs[] = { "1", "2", "3", "4", "5" };
        // Stats<String> strob = new
        Stats<String>(strs);
        // double x = strob.average();
        // System.out.println("strob average is " +
        v);
    }
}
```

```
class Stats<T extends Number>{
    T[] nums;
    Stats(T[] o) {
        nums = o;}
    double average() {
        double sum = 0.0;
        for(int i=0; i < nums.length; i++)
            sum += nums[i].doubleValue();
        return sum / nums.length;}
}
boolean sameAvg(Stats<T> ob) {
    if(average() == ob.average())
        return true;
    return false;
}
```

```
class BoundsDemo {
```

```
    public static void main(String args[]) {
        Integer inums[] = { 1, 2, 3, 4, 5 };
        Double dnums[] = { 1.1, 2.2, 3.3, 4.4, 5.5 };
        Stats<Integer> iob = new
            Stats<Integer>(inums);
        Stats<Double> dob = new
            Stats<Double>(dnums);
        if(iob.sameAvg(dob))
            System.out.println("Averages are the same.");
        else
            System.out.println("Averages differ.");
    }
```

```
class Stats<T extends Number>{
    T[] nums;
    Stats(T[] o) {
        nums = o;}
    double average() {
        double sum = 0.0;
        for(int i=0; i < nums.length; i++)
            sum += nums[i].doubleValue();
        return sum / nums.length;}
}
boolean sameAvg(Stats<?> ob) {
    if(average() == ob.average())
        return true;
    return false;
}
```

```
class BoundsDemo {
    public static void main(String args[]) {
        Integer inums[] = { 1, 2, 3, 4, 5 };
        Double dnums[] = { 1.1, 2.2, 3.3, 4.4, 5.5 };
        Stats<Integer> iob = new Stats<Integer>(inums);
        Stats<Double> dob = new Stats<Double>(dnums);
        if(iob.sameAvg(dob))
            System.out.println("Averages are the same.");
        else
            System.out.println("Averages differ.");
    }
}
```

- Question mark (?) is the wildcard in generics and represent an unknown type.
- The wildcard can be used as the type of a parameter, field, or local variable and sometimes as a return type.
- We can't use wildcards while invoking a generic method or instantiating a generic class.

What are Java Collections?



A Java Collection can be defined as a unified collection of objects.



Java Collection Framework



A Java Collection Framework is defined as a combined structure that is capable to store and apply operations to manipulate the data

A Java Collection Framework is defined as a combined structure that is capable to store and apply operations to manipulate the data

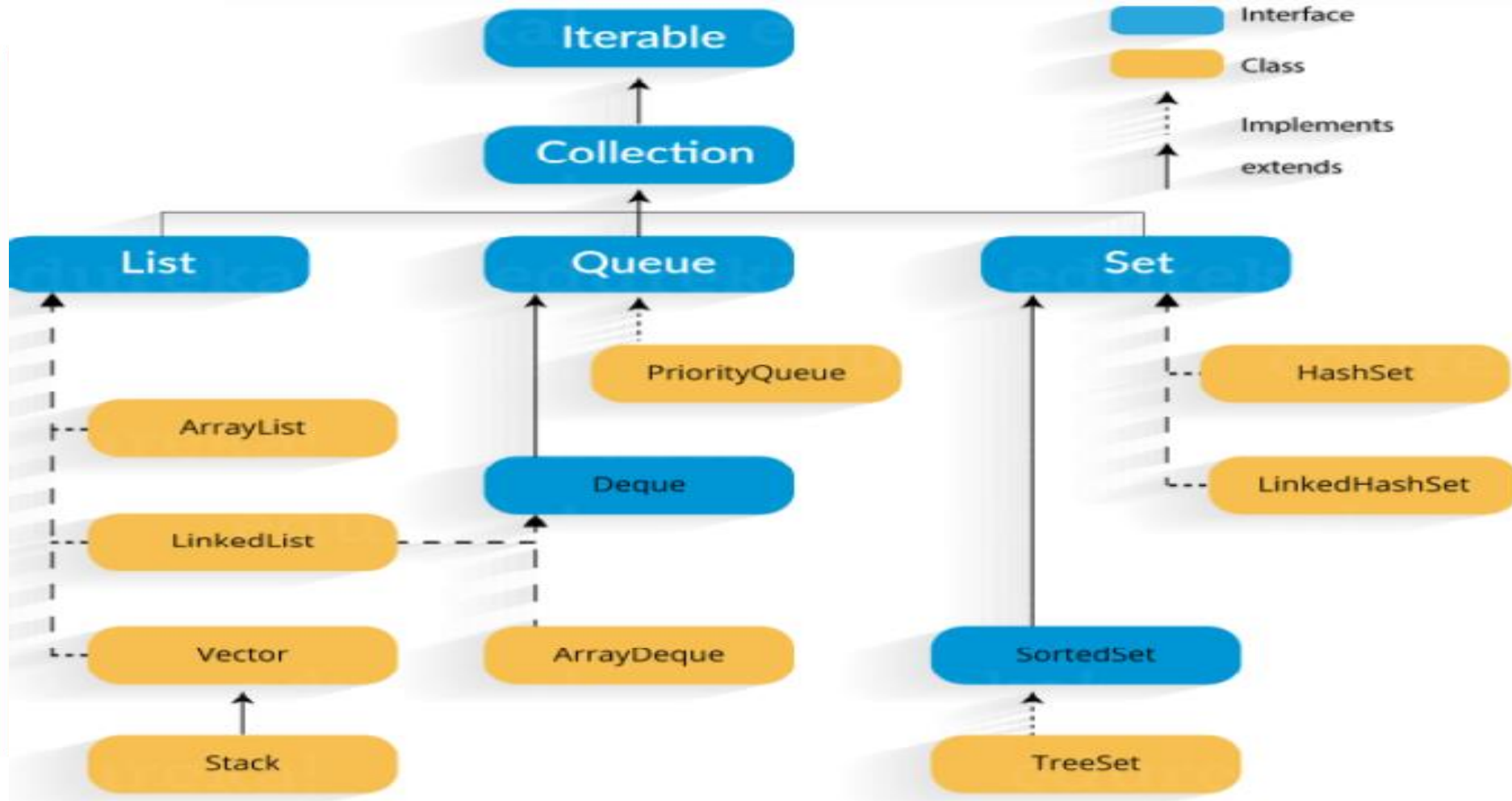
Interface

Classes

API

- Collections are like containers that group multiple items in a single unit. For example, a jar of chocolates, list of names, etc.
- The Java collections framework provides a set of interfaces and classes to implement various data structures and algorithms.

- The java.util package contains one of Java's most powerful subsystems: the Collections Framework
- The Collections Framework is a sophisticated hierarchy of interfaces and classes that provide state-of-the-art technology for managing groups of objects

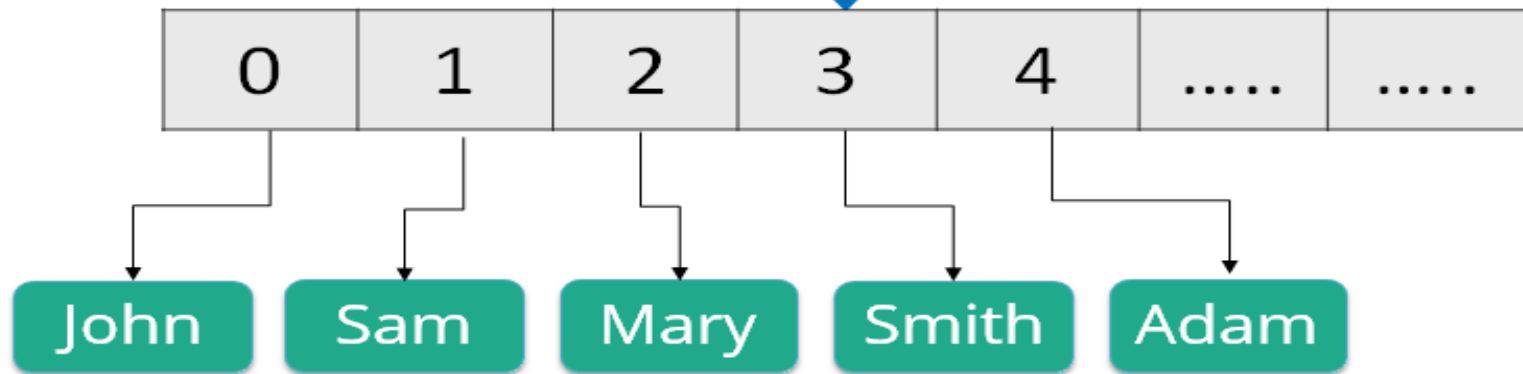


- A List is an ordered Collection of elements which may contain duplicates.
- It is an interface that extends the Collection interface.
- Lists are further classified into the following:
- ArrayList
- LinkedList
- Vectors

- Array list: ArrayList is the implementation of List Interface where the elements can be dynamically added or removed from the list.
- Also, the size of the list is increased dynamically if the elements are added more than the initial size
- The ArrayList class allows us to create resizable arrays
- Syntax:
- `ArrayList object = new ArrayList ();`
- ArrayList is a generic class that has this declaration:
- `class ArrayList<E>` - E specifies the type of objects that the list will hold
[ArrayList cannot store primitive types]

Java.util.ArrayList Size:5

elementData



Example

```
ArrayList<Integer> aint=new ArrayList<>();  
    System.out.println("size of integer arraylist" + aint.size());  
    for(int i=0;i<10;i++)  
        aint.add(i+10);  
    System.out.println("size of integer arraylist" + aint.size());  
    System.out.println("arraylist " + aint);
```

Output:

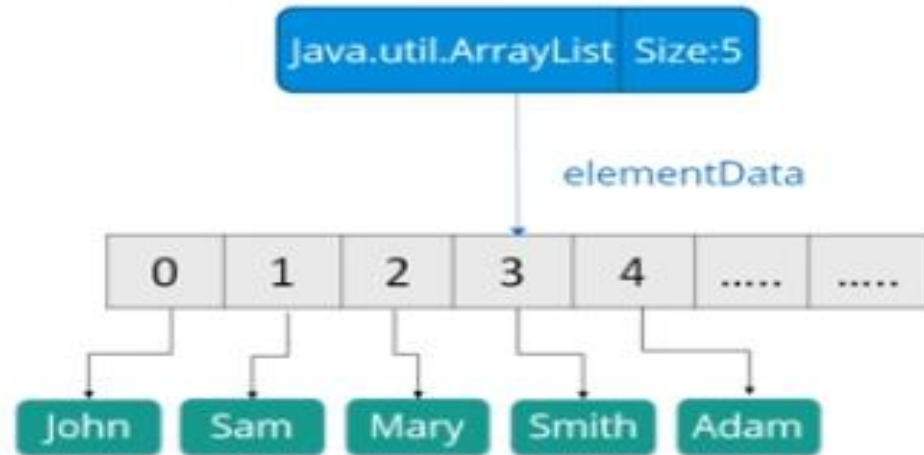
```
size of integer arraylist 0  
size of integer arraylist 10  
arraylist [10, 11, 12, 13, 14, 15, 16, 17, 18, 19]
```

Method	Description
boolean add(Collection c)	Appends the specified element to the end of a list.
void add(int index, Object element)	Inserts the specified element at the specified position.
void clear()	Removes all the elements from this list.
int lastIndexOf(Object o)	Return the index in this list of the last occurrence of the specified element, or -1 if the list does not contain this element.
Object clone()	Return a shallow copy of an ArrayList.
Object[] toArray()	Returns an array containing all the elements in the list.
void trimToSize()	Trims the capacity of this ArrayList instance to be the list's current size.

Hierarchy of Array List Class



ArrayList is a part of collection framework present in `java.util` package. It provides dynamic arrays in Java.



Internal Working of ArrayList



Adding elements to Array List



Creating a new array & moving the elements to new array



Array List Initialization



Capacity of Array List is Full



Size is increased to double

Size increases By 50%

Constructors of ArrayList

ArrayList()

ArrayList(Collection c)

ArrayList(int Capacity)

This constructor builds an empty array list.

Syntax:

```
ArrayList<E> myArray = new ArrayList<E>();
```

Constructor in ArrayList

ArrayList()

ArrayList(Collection c)

ArrayList(int Capacity)

This constructor builds an array list that is initialized with the elements of the collection c.

Syntax:

```
public boolean addAll(Collection c)
```

Method in ArrayList

`void add(int index, Object element)`

`void clear()`

`void trimToSize()`

`int indexOf(Object O)`

`Object clone()`

`Object[] to Array()`

`Object remove(int index)`

This method is used to insert a specific element at a specific position index in a list.

Example:

```
ArrayList<String> al=new ArrayList<String>();  
al.add("Jino");  
al.add("Piyush");  
al.add("Pramod");
```


void add(int index, Object element)

void clear()

void trimToSize()

int indexOf(Object O)

Object clone()

Object[] toArray()

Object remove(int index)

int size()

This method is used to remove all the elements from any list.

Example:

```
ArrayList<String> al=new ArrayList<String>();  
al.add("abc");  
al.add("xyz");  
System.out.println("ArrayList before clear: "+al);  
al.clear();  
System.out.println("ArrayList after clear: "+al);
```

`void add(int index, Object element)``void clear( )``void trimToSize()``int indexOf(Object O)``Object clone()``Object[] to Array()``Object remove(int index)``int size()`

The **trimToSize()** method in Java trims the capacity of an ArrayList instance to be the list's current size. This method is used to trim an ArrayList instance to the number of elements it contains.

Example:

```
ArrayList<Integer> al = new ArrayList<Integer>(9);
```

```
al.add(2);
```

```
al.add(4);
```

```
al.add(5);
```

```
al.trimToSize();
```

```
System.out.println("The List elements are:");
```

`void add(int index, Object element)``void clear()``void trimToSize()``int indexOf(Object O)``Object clone()``Object[] to Array()``Object remove(int index)``int size()`

This method returns the index of the **first occurrence** of the specified element in this list, or **-1** if this list does not contain the element.

Example:

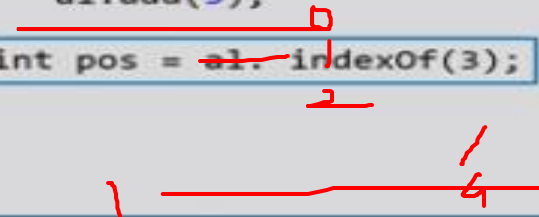
```
ArrayList<Integer> al = new ArrayList<Integer>(9);
```

```
al.add(2);
```

```
al.add(4);
```

```
al.add(5);
```

```
int pos = al.indexOf(3);
```



`void add(int index, Object element)``void clear()``void trimToSize()``int indexOf(Object O)``Object clone()``Object[] to Array()``Object remove(int index)``int size()`

Returns a shallow copy of this ArrayList.

Example:

```
ArrayList<String> al=new ArrayList<String>();
```

```
Object cloneList; //Added 2 elements
```

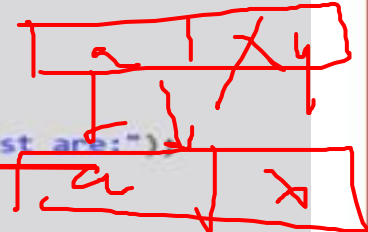
```
al.add("abc");
```

```
al.add("xyz");
```

```
System.out.println("Elements are: ");
```

```
cloneList = al.clone();
```

```
System.out.println("Elements in cloneList are:");
```



`void add(int index, Object element)``void clear()``void trimToSize()``int indexOf(Object O)``Object clone()``Object[] toArray()``Object remove(int index)``int size()`

Returns an array containing all of the elements in this list in the correct order; the runtime type of the returned array is that of the specified array.

Example:

```
arrayList.add("element_1");  
arrayList.add("element_2");  
arrayList.add("element_3");  
arrayList.add("element_4");
```

```
Object[] objArray = arrayList.toArray();
```

void add(int index, Object element)

void clear()

void trimToSize()

int indexOf(Object O)

Object clone()

Object[] to Array()

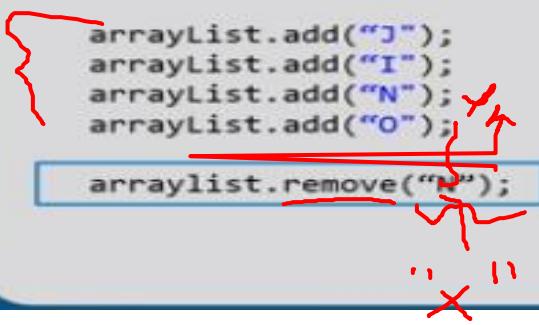
Object remove(int index)

int size()

java.util.ArrayList.remove(Object) method removes the first occurrence of the specified element from this list, if it is present. If the list does not contain the element, it is unchanged.

Example:

```
arrayList.add("J");  
arrayList.add("I");  
arrayList.add("N");  
arrayList.add("O");  
arraylist.remove("N");
```



`void add(int index, Object element)``void clear()``void trimToSize()``int indexOf(Object O)``Object clone()``Object[] to Array()``Object remove(int index)``int size()`

It returns the number of elements in this list i.e the size of the list.

Example:

Declaration: `public int size()`

```
arrayList.add("J");  
arrayList.add("I");  
arrayList.add("N");  
arrayList.add("O");
```

```
int asize = arrayList.size();
```

4

- Used to obtain actual array from ArrayList
- toArray() - defined by Collection
- What is the need?
- To obtain faster processing times for certain operations
- To pass an array to a method that is not overloaded to accept a collection
- To integrate collection-based code with legacy code that does not understand collections
- Sample Program

ensureCapacity()

- Although the capacity of an ArrayList object increases automatically as objects are stored in it, you can increase the capacity of an ArrayList object manually by calling ensureCapacity()
- By increasing its capacity once, at the start, you can prevent several reallocations later
- void ensureCapacity(int cap)
- If you know in advance that you will be storing many more items in the collection than it can currently hold

Advantages of ArrayList Over Array



- Iterator/ListIterator is an interface that iterates over the elements. It is used to traverse the list and modify the elements
- Iterator and ListIterator are generic interfaces which are declared as shown here:
- interface Iterator<E> and interface ListIterator<E>
- public boolean hasNext() – This method returns true if the iterator has more elements.
- public Object next() – It returns the element and moves the cursor pointer to the next element. Throws NoSuchElementException if there is not a next element
- public void remove() – This method removes the last elements returned by the iterator.
- ListIterator also provides hasPrevious(), previous(), nextIndex(), previousIndex(), set() methods

1. Obtain an iterator to the start of the collection by calling the collection's `iterator( )` method
 2. Set up a loop that makes a call to `hasNext( )` and the loop iterate as long as `hasNext( )` returns true
 3. Within the loop, obtain each element by calling `next( )`
- For collections that implement `List`, you can also obtain an iterator by calling `listIterator( )`

For Each Loop as an Alternative

Create an array list for integers

```
ArrayList<Integer> vals = new ArrayList<Integer>();
```

```
// Add values to the array list
```

```
vals.add(1);
```

```
vals.add(2);
```

```
vals.add(3);
```

```
vals.add(4);
```

```
vals.add(5);
```

```
// Use for loop to display the values.
```

```
System.out.print("Contents of vals: ");
```

```
for(Integer v : vals)
```

```
    System.out.print(v + " ");
```

Example

- Write a program to create a class student with name and marks as private members. Define getters and setters.
- Create an ArrayList to store objects of student class.
- Create a method which takes ArrayList as input and returns the average marks of all students.
- Create another method which takes input as ArrayList of student and returns a new arraylist which contains the student with below average marks.

```
import java.util.ArrayList;
import java.util.Iterator;
class Student{
    private String name;
    private int marks;
    public Student(String name, int marks){
        this.name = name;
        this.marks = marks;
    }
    public String getName(){
        return this.name;
    }
    public int getMarks(){
        return this.marks;
    }
}
```

```
public class ArrayListofObject{
    public static void main(String args[]){
        //Create objects of Student class
        Student s1 = new Student("Alex", 80);
        Student s2 = new Student("Kris", 60);
        Student s3 = new Student("Peter", 90);
        //Create an ArrayList of objects Student
        ArrayList<Student> aListStudents = new
        ArrayList<Student>();
        //add Objects of Student to ArrayList
        aListStudents.add(s1);
        aListStudents.add(s2);
        aListStudents.add(s3);
        //display ArrayList objects
        for(Student student : aListStudents){
            System.out.println("Name: " +
            student.getName() + ", Marks:
            "+student.getMarks());
        }
    }
}
```

```
    }
    double avg=calculateaverage(aListStudents);
    System.out.println("Average of Class is " +
    avg);
    ArrayList<Student>
    BAVG=belowavg(aListStudents);
    System.out.println("List of Students Below
    Average");
    for(Student student : BAVG){
        System.out.println("Name: " +
        student.getName() + ", Marks:
        "+student.getMarks());
    }
}
```



```
static double  
calculateaverage(ArrayList<Student> s)  
{  
    double avg=0;  
    int count=0;  
    Iterator itr=s.iterator();  
    while(itr.hasNext()) {  
        Student s1=(Student)itr.next();  
        avg += s1.getMarks();  
        count++;  
    }  
    avg = avg/count;  
    return avg;  
}
```

```
static ArrayList<Student>  
belowavg(ArrayList<Student> s){
```

```
    ArrayList<Student> bavg=new  
    ArrayList<Student>();  
    double avg=calculateaverage(s);  
    for(Student s1 : s ) {  
        if(s1.getMarks()<avg)  
            bavg.add(s1);  
    }  
    return bavg;  
}
```

Sorting an ArrayList

```
public class Integercomparable {  
  
    public static void main(String  args[]){  
        ArrayList<Integer> list = new  
            ArrayList<Integer>();  
        list.add(50);    list.add(15);  
        list.add(450);    list.add(550);  
        list.add(200);  
  
        System.out.println(list);  
        Collections.sort(list);  
  
        System.out.println("ascending list: " + list)
```

Sorting an ArrayList in Descending Order

```
import java.util.*;
class DescComp implements Comparator<Integer>{
    public int compare(Integer o1, Integer o2) {
        if(o1<o2)
            return 1;
        else
            if(o1>o2)
                return -1;
            else
                return 0;
    }
}
```

```
public class integercomparable {
    public static void main(String args[]){
        ArrayList<Integer> list = new
        ArrayList<Integer>();
        list.add(50);    list.add(15);
        list.add(450);    list.add(550);
        list.add(200);
        System.out.println(list);
        Collections.sort(list);
        System.out.println("ascending list: " + list);
        Collections.sort(list,new DescComp());
        System.out.println("descending list: " + list);
    } }
```

Sorting an object using Comparable

```
class item implements Comparable<item>{
    int rate;
    int quantity;
    item(int r, int q){
        rate=r;
        quantity=q;
    }
    void display(){
        System.out.print("rate " + rate);
        System.out.println("Quantity " + quantity);
    }
    public int compareTo(item o) {
        if(rate > o.rate)
            return 1;
        else if (rate < o.rate)
            return -1;
        else
            return 0;
    }
}
```

```
public class arraylist {
    public static void main(String args[])
    {
        ArrayList<item> ArrItem=new
        ArrayList<item>(5);
        ArrItem.add(new item(7,10));
        ArrItem.add(new item(5,10));
        ArrItem.add(new item(6,20));

        for(item i:ArrItem){
            i.display();
        }
        Collections.sort(ArrItem);
        for(item i:ArrItem)
        {
            i.display();
        }
    }
}

sort(List<T> list) // T extends Comparable
sort(List<T> list, Comparator c)
```

DO it yourself (NamesArray)

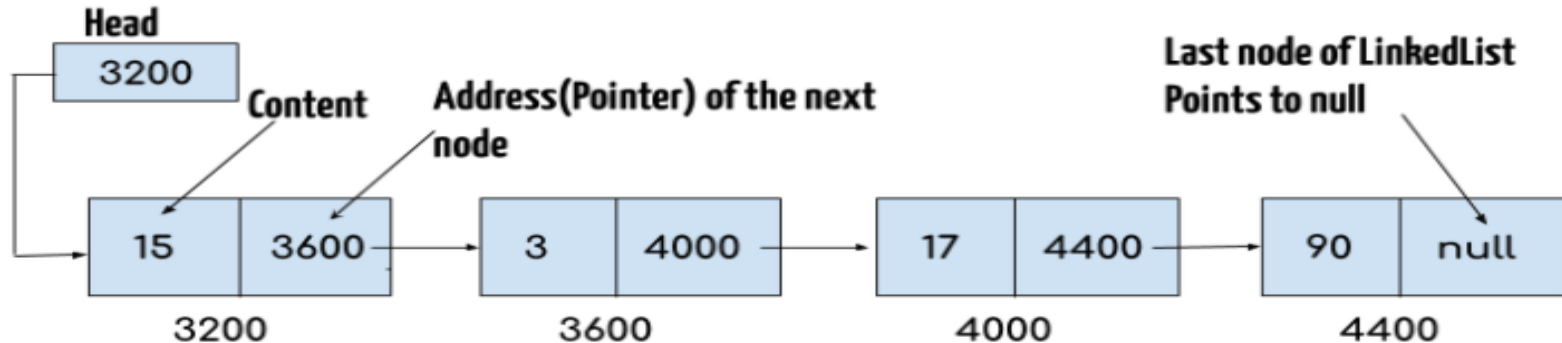
- 1) Ask the user how many names they will enter.
- 2) Then ask for those names, storing them in an ArrayList. Display the list.
- 3) Ask the user to choose one of the names. Delete this person from the list. Display the list.
- 4) Insert your own name so that it's the 2nd name in the list. Display the list.
- 5) Then add the name "N Modi" to the end of the list. Display the list.
- 6) Then, ask the user to choose another name to delete, as well as what name will replace it. Replace the name. Display the list.
- 7) Then, ask how old each person on the list is (ex: "How old is N Modi?"). Insert each person's age immediately after that person's name in the List. Display the list.

- Similar to arrays in Java, LinkedList is a linear data structure.
- LinkedList elements are not stored in contiguous locations like arrays, they are linked with each other using pointers.
- Each element of the LinkedList has the reference(address/pointer) to the next element of the LinkedList.

LinkedList representation

- Each element in the LinkedList is called the Node.
- Each Node of the LinkedList contains two items:
 - 1) Content of the element
 - 2) Pointer/Reference to the Next Node in the LinkedList

This is how a LinkedList looks:



- Creating a Java LinkedList
- `LinkedList<Type> linkedList = new LinkedList<>();`
- `// create Integer type linked list`
- `LinkedList<Integer> linkedList = new LinkedList<>();`
- `// create String type linked list`
- `LinkedList<String> linkedList = new LinkedList<>();`

- provides various methods that allow us to perform different operations in linked lists
- Add elements
- Access elements
- Change elements
- Remove elements

Method	Description
addFirst()	Adds an item to the beginning of the list.
addLast()	Add an item to the end of the list
removeFirst()	Remove an item from the beginning of the list.
removeLast()	Remove an item from the end of the list
getFirst()	Get the item at the beginning of the list
getLast()	Get the item at the end of the list

LinkedList Example

```
public static void main(String a[]){
    LinkedList<Integer> ll=new LinkedList<Integer>();
    for(int i=1;i<5;i++){
        ll.add(i+10);
    } // 11,12,13,14,15
    ll.addFirst(0); //0, 11,12,13,14,15
    ll.addLast(5); //0, 11,12,13,14,15,5
    System.out.println(ll); //[0, 11,12,13,14,15,5]
    System.out.println(ll.get(2)); //returns 12
    System.out.println(ll.getFirst()); //returns 0
    System.out.println(ll.getLast()); //returns 5
    System.out.println(ll.offer(20)); //adds the specified elem at tail == 0,
    11,12,13,14,15,5,20
}
```

```
System.out.println(ll.offerFirst(-1)); //-1, 0, 11,12,13,14,15,5,20
    System.out.println(ll.offerLast(30)); //-1, 0, 11,12,13,14,15,5,20,30
    System.out.println(ll);[-1, 0, 11,12,13,14,15,5,20,30]
System.out.println(ll.indexOf(20)); //returns 8
System.out.println(ll.lastIndexOf(4)); //returns -1
    System.out.println(ll.remove()); //retrieves and removes head; returns head
    System.out.println(ll); // [0, 11,12,13,14,15,5,20,30]
System.out.println(ll.remove((Object)5)); //// [0, 11,12,13,14,15,20,30]
System.out.println(ll);
Iterator itr=ll.descendingIterator();
//prints in reverse order
while(itr.hasNext())
{
    System.out.println(itr.next());
}
```

```
ListIterator litr=ll.listIterator(3);
    System.out.println("Printing before index 3");
    while(litr.hasPrevious())
        System.out.println(litr.previous()); //0,11,12

    System.out.println("Print using List Iterator");
    while(litr.hasNext())
        System.out.println(litr.next()); //0, 11,12,13,14,15,20,30

    Collections.reverse(ll);
}}
```

- In the ArrayList, we learned that Arrays store items as an ordered collection, and you have to access them with an index number (int type).
- A HashMap however, store items in "key/value" pairs, and you can access them by an index of another type (e.g. a String).
- One object is used as a key (index) to another object (value). It can store different types: String keys and Integer values, or the same type, like: String keys and String values

HashMap
key -> value

Simple Example

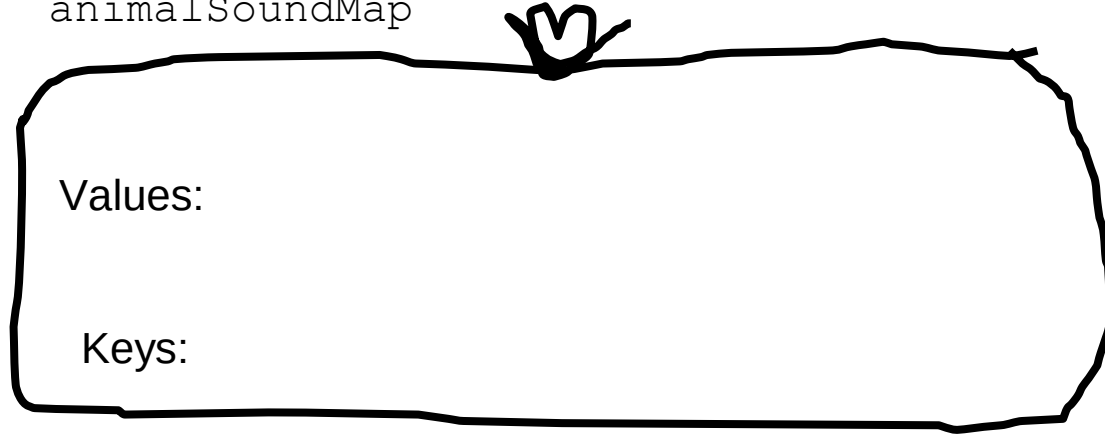
1. Make a new HashMap of animal sounds
2. Add elements:
Put [key = "dog", value = "bark"]
Put [key="cat", value="meow"]
Put [key="seal", value="ow ow ow"]
3. Get elements:
Get [key = "dog"]

Simple Example

1. Make a new HashMap of animal sounds
2. Add elements:
Put [key = "dog", value = "bark"]
Put [key="cat", value="meow"]
Put [key="seal", value="ow ow ow"]
3. Get elements:
Get [key = "dog"]

Simple Example

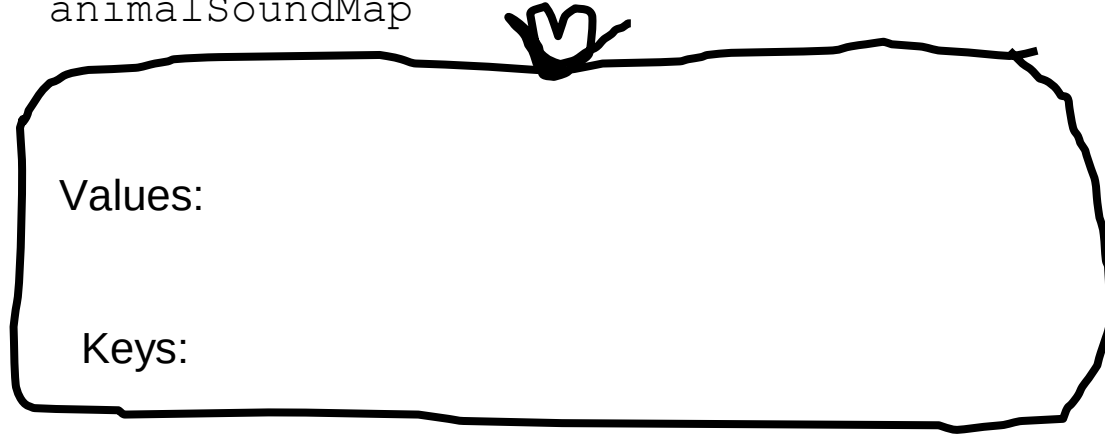
animalSoundMap



1. **Make a new HashMap of animal sounds**
2. Add elements:
Put [key = "dog", value = "bark"]
Put [key="cat", value="meow"]
Put [key="seal", value="ow ow ow"]
3. Get elements:
Get [key = "dog"]

Simple Example

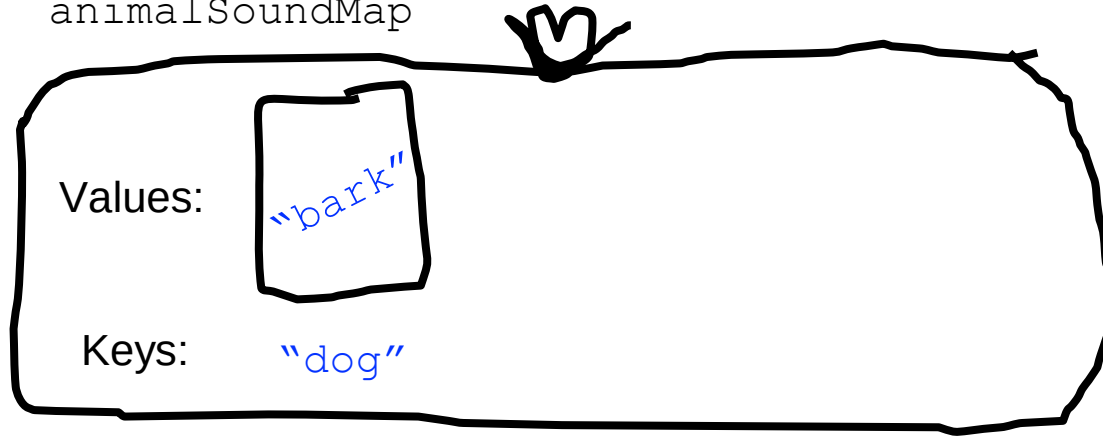
animalSoundMap



1. Make a new HashMap of animal sounds
2. Add elements:
`Put [key = "dog", value = "bark"]`
`Put [key="cat", value="meow"]`
`Put [key="seal", value="ow ow ow"]`
3. Get elements:
`Get [key = "dog"]`

Simple Example

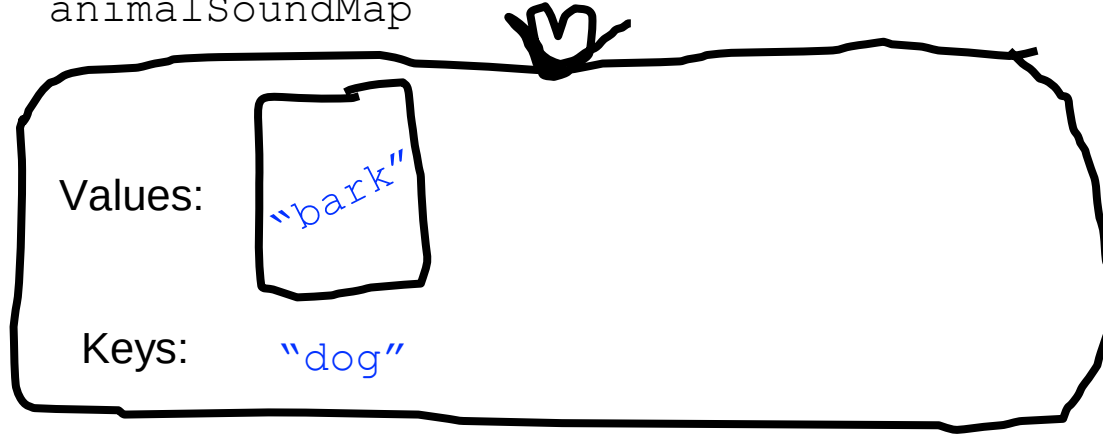
animalSoundMap



1. Make a new HashMap of animal sounds
2. Add elements:
Put [key = "dog", value = "bark"]
Put [key="cat", value="meow"]
Put [key="seal", value="ow ow ow"]
3. Get elements:
Get [key = "dog"]

Simple Example

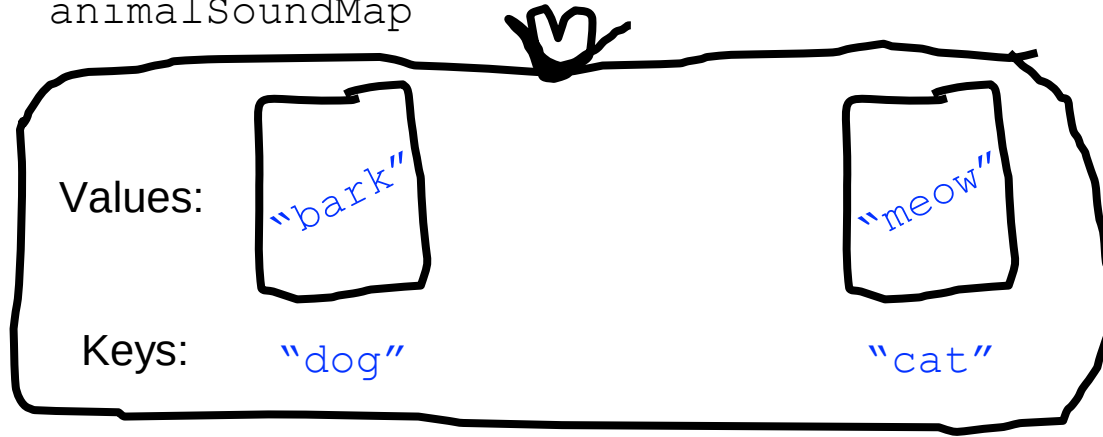
animalSoundMap



1. Make a new HashMap of animal sounds
2. Add elements:
Put [key = "dog", value = "bark"]
Put [key="cat", value="meow"]
Put [key="seal", value="ow ow ow"]
3. Get elements:
Get [key = "dog"]

Simple Example

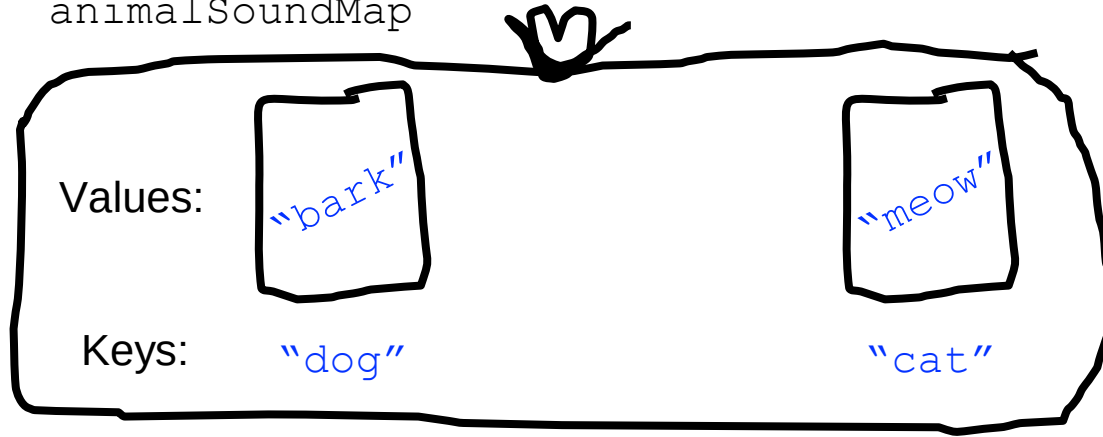
animalSoundMap



1. Make a new HashMap of animal sounds
2. Add elements:
Put [key = "dog", value = "bark"]
Put [key="cat", value="meow"]
Put [key="seal", value="ow ow ow"]
3. Get elements:
Get [key = "dog"]

Simple Example

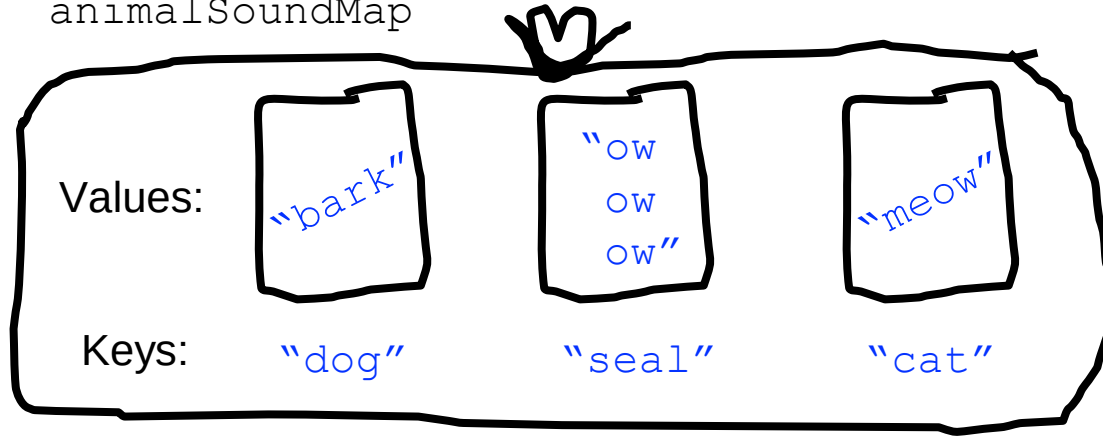
animalSoundMap



1. Make a new HashMap of animal sounds
2. Add elements:
Put [key = "dog", value = "bark"]
Put [key="cat", value="meow"]
Put [key="seal", value="ow ow ow"]
3. Get elements:
Get [key = "dog"]

Simple Example

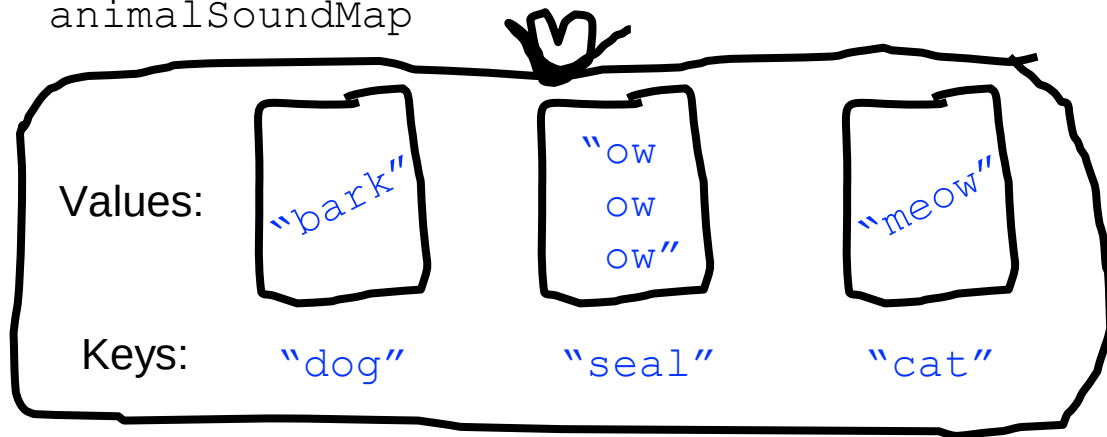
animalSoundMap



1. Make a new HashMap of animal sounds
2. Add elements:
Put [key = "dog", value = "bark"]
Put [key="cat", value="meow"]
Put [key="seal", value="ow ow ow"]
3. Get elements:
Get [key = "dog"]

Simple Example

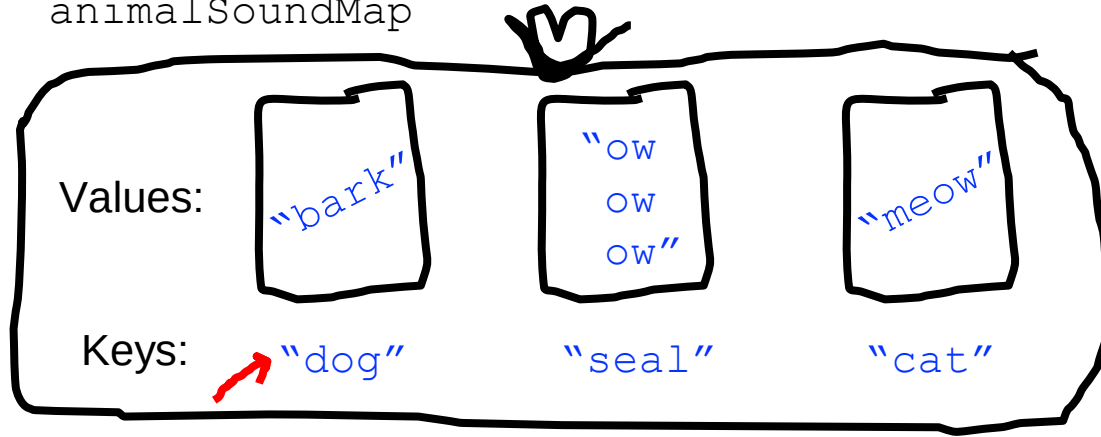
animalSoundMap



1. Make a new HashMap of animal sounds
2. Add elements:
Put [key = "dog", value = "bark"]
Put [key="cat", value="meow"]
Put [key="seal", value="ow ow ow"]
3. Get elements:
Get [key = "dog"]

Simple Example

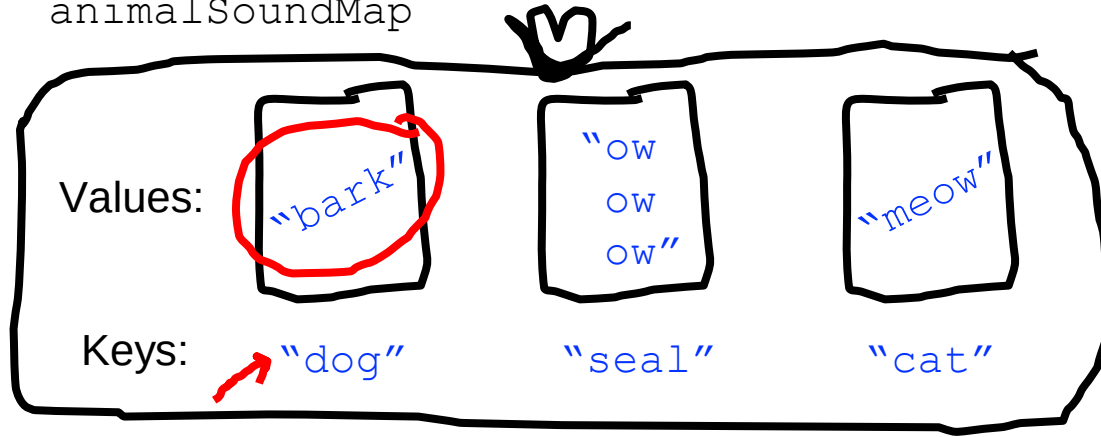
animalSoundMap



1. Make a new HashMap of animal sounds
2. Add elements:
Put [key = "dog", value = "bark"]
Put [key="cat", value="meow"]
Put [key="seal", value="ow ow ow"]
3. Get elements:
Get [key = "dog"]

Simple Example

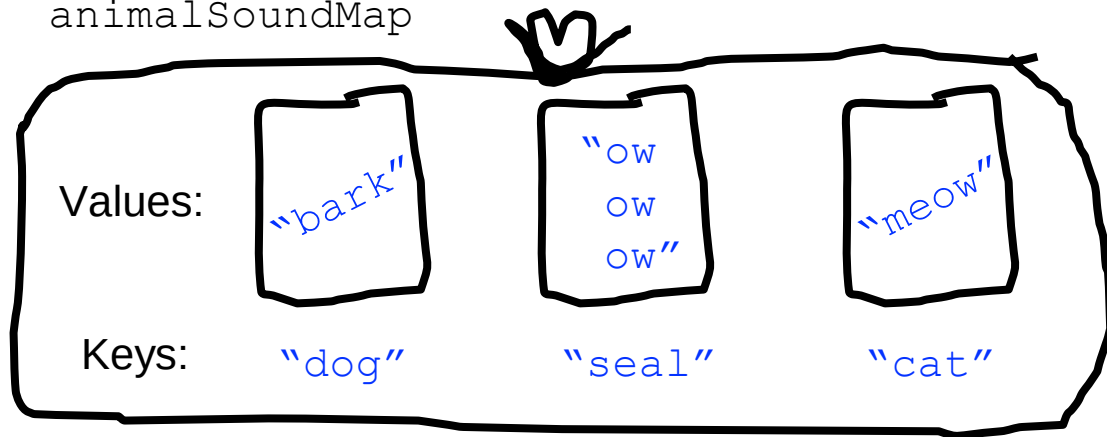
animalSoundMap



1. Make a new HashMap of animal sound
2. Add elements:
Put [key = "dog", value = "bark"]
Put [key="cat", value="meow"]
Put [key="seal", value="ow ow ow"]
3. Get elements:
Get [key = "dog"]

Simple Example

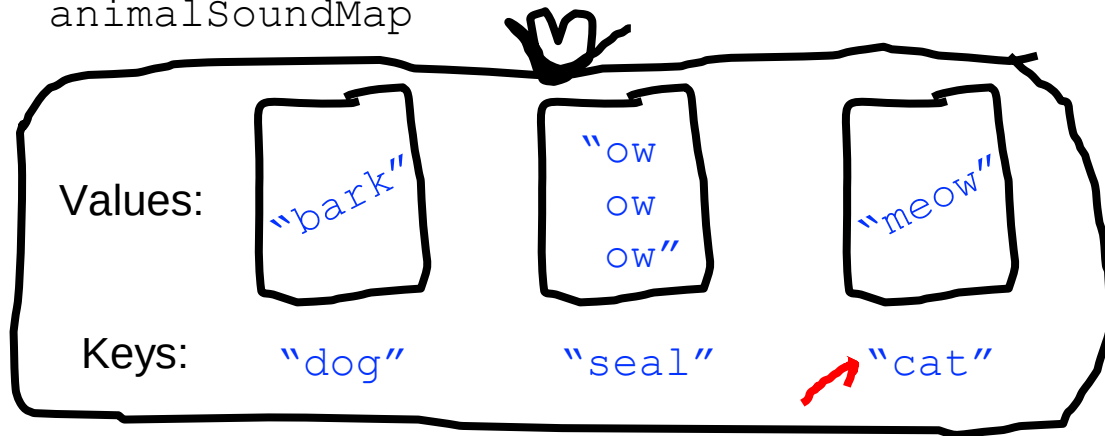
animalSoundMap



1. Make a new HashMap of animal sound
2. Add elements:
Put [key = "dog", value = "bark"]
Put [key="cat", value="meow"]
Put [key="seal", value="ow ow ow"]
3. Get elements:
Get [key = "cat"]

Simple Example

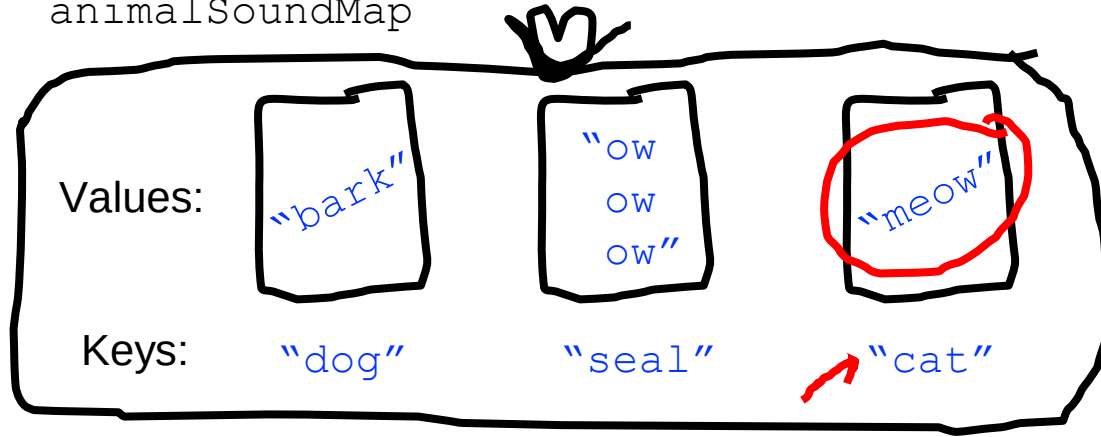
animalSoundMap



1. Make a new HashMap of animal sound
2. Add elements:
Put [key = "dog", value = "bark"]
Put [key="cat", value="meow"]
Put [key="seal", value="ow ow ow"]
3. Get elements:
Get [key = "cat"]

Simple Example

animalSoundMap



1. Make a new HashMap of animal sound
2. Add elements:
Put [key = "dog", value = "bark"]
Put [key="cat", value="meow"]
Put [key="seal", value="ow ow ow"]
3. Get elements:
Get [key = "cat"]

My First Map

```
HashMap<String, String> animalSoundMap =  
    new HashMap<String, String>();
```

My First Map

Key Type



Value Type



```
HashMap<String, String> animalSoundMap =  
    new HashMap<String, String>();
```


My First Map

```
HashMap<String, String> animalSoundMap =  
    new HashMap<String, String>();
```

My First Map

```
HashMap<String, String> animalSoundMap =  
    new HashMap<String, String>();
```

My First Map

```
HashMap<String, String> animalSoundMap =  
    new HashMap<String, String>();  
  
animalSoundMap.put("dog", "bark");
```

My First Map

```
HashMap<String, String> animalSoundMap =  
    new HashMap<String, String>();  
  
animalSoundMap.put("dog", "bark");
```

My First Map

```
HashMap<String, String> animalSoundMap =  
    new HashMap<String, String>();  
  
animalSoundMap.put("dog", "bark");
```

My First Map

```
HashMap<String, String> animalSoundMap =  
    new HashMap<String, String>();  
  
animalSoundMap.put("dog", "bark");
```

My First Map

```
HashMap<String, String> animalSoundMap =  
    new HashMap<String, String>();  
  
animalSoundMap.put("dog", "bark");  
  
animalSoundMap.get("dog");
```

My First Map

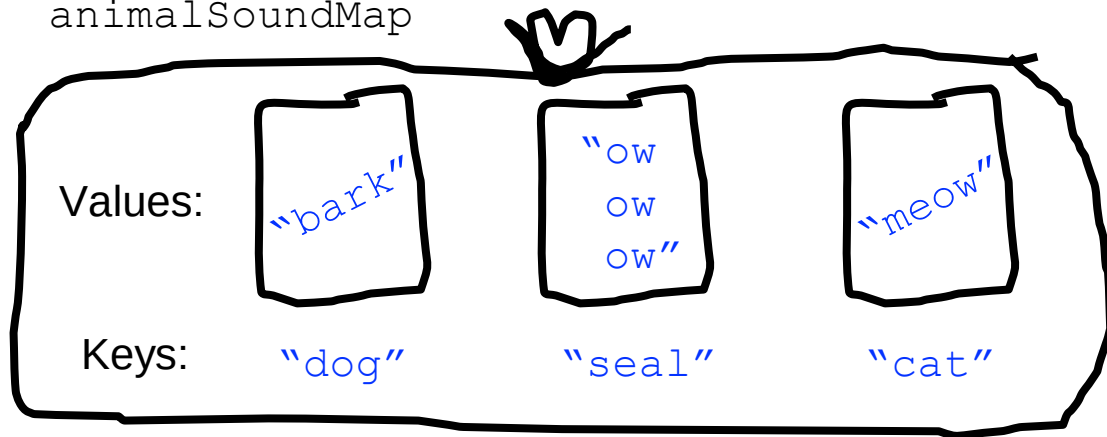
```
HashMap<String, String> animalSoundMap =  
    new HashMap<String, String>();  
  
animalSoundMap.put("dog", "bark");  
  
animalSoundMap.get("dog");
```


My First Map

```
HashMap<String, String> animalSoundMap =  
    new HashMap<String, String>();  
  
animalSoundMap.put("dog", "bark");  
  
animalSoundMap.get("dog");
```

My First Map

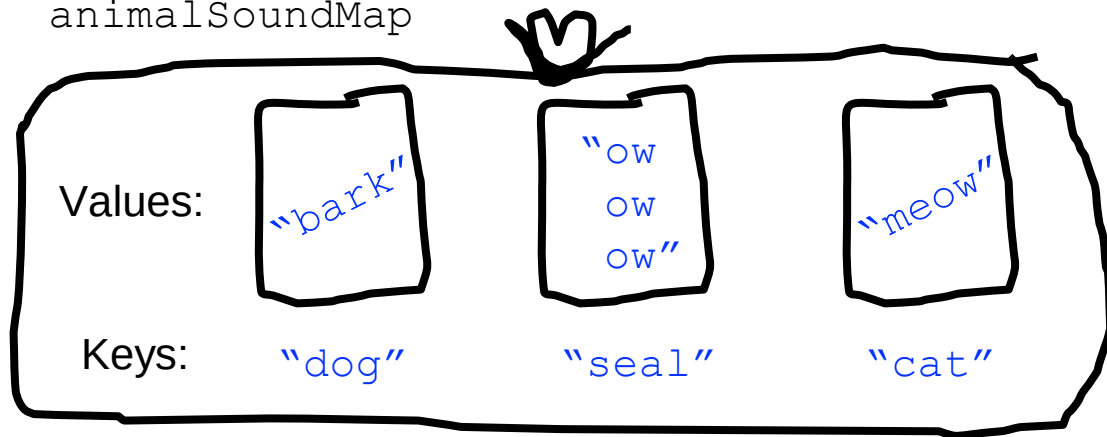
animalSoundMap



1. Make a new HashMap of animal sound
2. Add elements:
Put [key = "dog", value = "bark"]
Put [key="cat", value="meow"]
Put [key="seal", value="ow ow ow"]
3. Get elements:
Get [key = "dog"]

My First Map

animalSoundMap



// 1. Make a new map

```
HashMap<String, String> animalSoundMap =
```

new

```
HashMap<String, String>();
```

// 2. Put things into the map

```
animalSoundMap.put("dog", "woof");
```

```
animalSoundMap.put("cat", "meow");
```

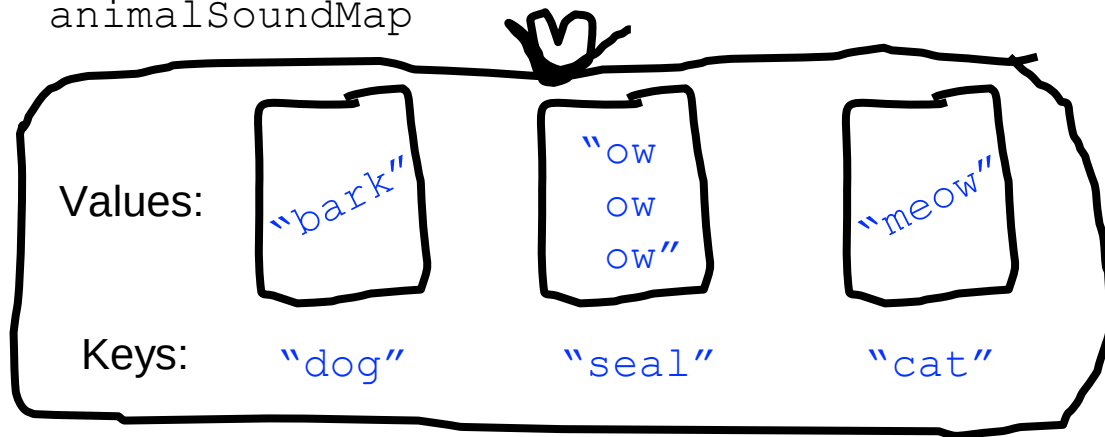
```
animalSoundMap.put("seal", "ow ow ow");
```

// 3. Get things out of the map

```
animalSoundMap.get("dog"); // "woof"
```

My First Map

animalSoundMap



// 1. Make a new map

```
HashMap<String, String> animalSoundMap =
```

new

```
HashMap<String, String>();
```

// 2. Put things into the map

```
animalSoundMap.put("dog", "woof");
```

```
animalSoundMap.put("cat", "meow");
```

```
animalSoundMap.put("seal", "ow ow ow");
```

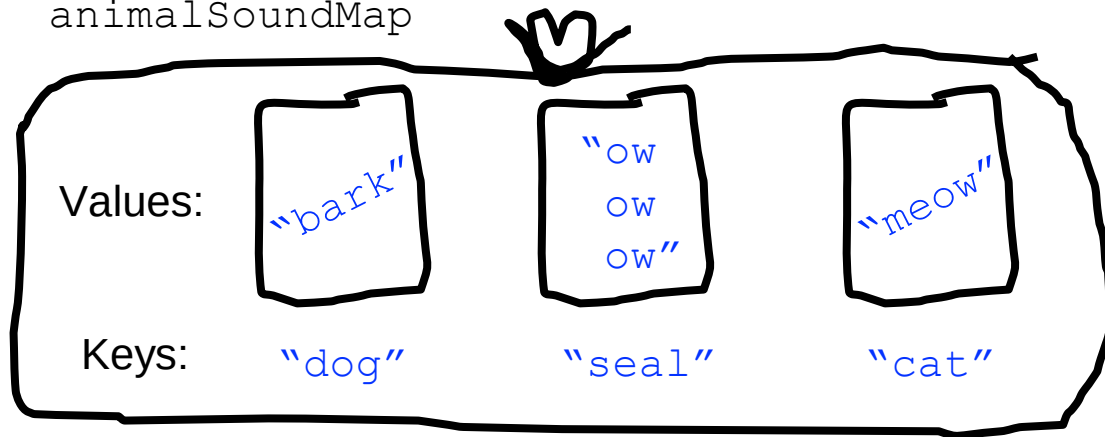
// 3. Get things out of the map

```
animalSoundMap.get("dog"); // "woof"
```

```
animalSoundMap.get("fox"); // ?
```

My First Map

animalSoundMap



// 1. Make a new map

```
HashMap<String, String> animalSoundMap =
```

new

```
HashMap<String, String>();
```

// 2. Put things into the map

```
animalSoundMap.put("dog", "woof");
```

```
animalSoundMap.put("cat", "meow");
```

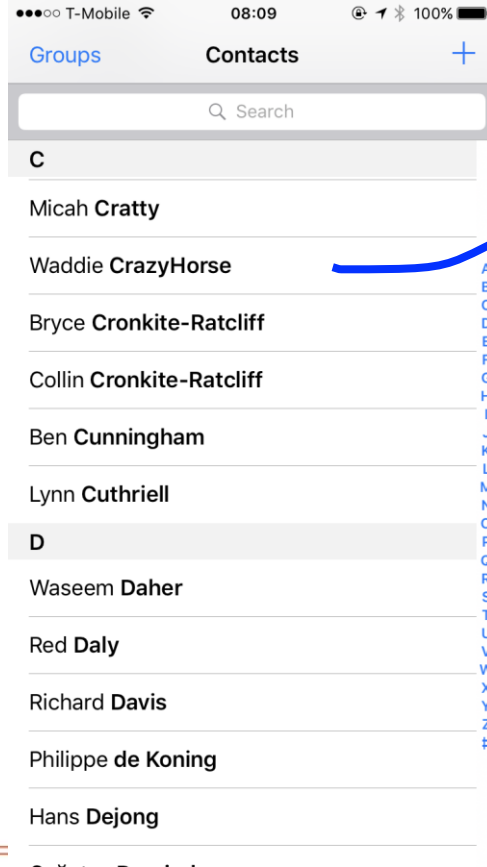
```
animalSoundMap.put("seal", "ow ow ow");
```

// 3. Get things out of the map

```
animalSoundMap.get("dog"); // "woof"
```

```
animalSoundMap.get("fox"); // null
```

Phone Book



6701678

HashMaps on one slide

1. Make a HashMap

```
HashMap<keyType><valueType> myMap =  
                                new HashMap<keyType><valueType>();
```

2. Put and get values into a map

```
myMap.put(key, value);  
myMap.get(key) // returns the corresponding value
```

3. Some useful other methods

```
int size = myMap.size();  
myMap.containsKey(key); // returns true or false if key is in map  
myMap.keySet();  
myMap.remove(key); // make like a tree and leave!
```

4. Iterate using a foreach loop

```
for(keyType key : myMap.keySet()){ // not ordered  
    myMap.get(key); // do something with the key/value pair  
}
```

Methods of HashMap

METHODS IN HASHMAP

<code>put(K key,V value)</code>	Inserts an entry in the map.
<code>putAll(Map map)</code>	Inserts a specified map in the map.
<code>putIfAbsent(K key, V value)</code>	Inserts the entry only when the key is not present.
<code>remove(K key)</code>	Deletes the entry for the specified key.
<code>remove(K key,V value)</code>	Deletes the entry for the specified key and value.
<code>clear()</code>	Removes all the mappings from the map.
<code>isEmpty()</code>	Returns true if the map has no key-value mappings.
<code>size()</code>	Returns the number of key-value mappings.
<code>keySet()</code>	Returns a set of keys in a hashmap.
<code>values()</code>	Returns a set of values in a hashmap.
<code>entrySet()</code>	Returns a set of entries(K, V) in a hashmap.
<code>get(K key)</code>	Returns the value associated with the given key.

[HashMap Examples](#)

Java HashMap Example

Create a HashMap object called capitalCities that will store String keys and String values:

- `import java.util.HashMap; // import the HashMap class`
- `HashMap<String, String> capitalCities = new HashMap<String, String>();`

Add Items to HashMap

Create a HashMap object called capitalCities that will store String keys and String values:

- `import java.util.HashMap; // import the HashMap class`
- `HashMap<String, String> capitalCities = new HashMap<String, String>();`

Java HashMap Example

Create a HashMap object called capitalCities that will store String keys and String values:

- `import java.util.HashMap; // import the HashMap class`
- `HashMap<String, String> capitalCities = new HashMap<String, String>();`

- HashSet in Java is one of the most important class of Java Collection hierarchy
- It is typically used to store unique values in an unordered manner
- `java.util.HashSet` class is a member of the Java collections framework which inherits the `AbstractSet` class and implements the `Set` interface.

{a,a,a,b,b,c,d,e,e,e}

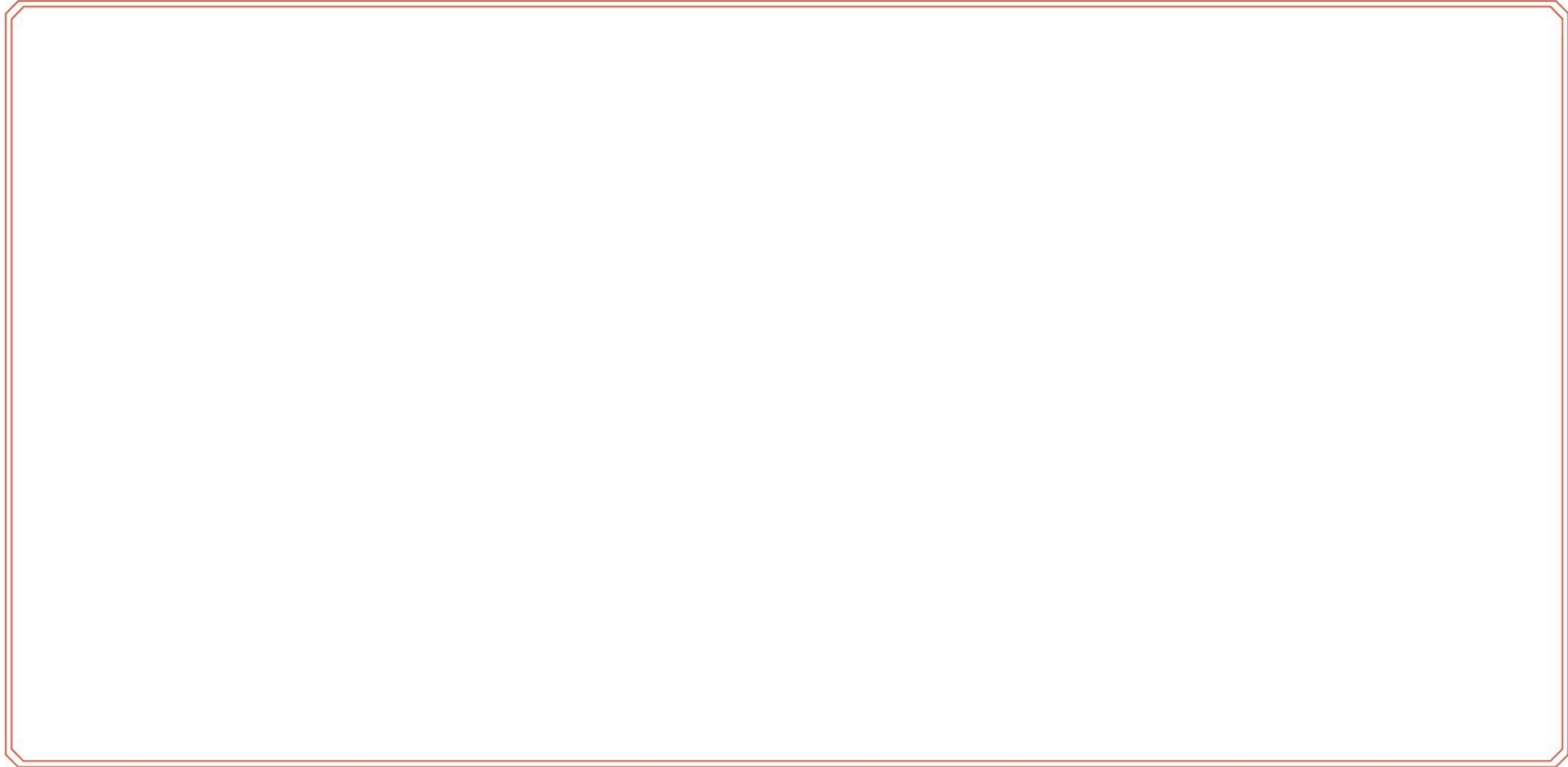


H A S H S E T



{a,b,c,d,e}

- A HashSet in Java does not allow duplicate values. If try to add by mistake, add() method returns false. No compile time error or runtime exception is reported
- It can contain null values
- HashSet doesn't follow the insertion order for storing the data rather it makes use of the hashcode for indexing the values within
- HashSet is the implementation class of Set interface
- Underlying implementation is Hashtable
- Best choice for Search operation





Constructors of java.util.HashSet Class

Constructor	Description
<i>HashSet ()</i>	This is the default constructor of HashSet class
<i>HashSet (int capacity)</i>	This constructor is used to initialize the initial capacity of the hash set. The capacity can grow dynamically with the addition of new elements
<i>HashSet (int capacity, float loadCapacity)</i>	This constructor is used to initialize the initial capacity of the hash set along with the load capacity
<i>HashSet (Collection c)</i>	This constructor is used to initialize the hash set by using the elements from Collection c

Methods of java.util.HashSet Class

Method	Description
<i>boolean add(Object obj)</i>	This method helps in adding a specified element into the HashSet only if it is not present
<i>void clear()</i>	This method helps in removing all of the elements from HashSet
<i>Object clone()</i>	This method returns a shallow copy of the HashSet instance rather than clones of the HashSet elements
<i>boolean contains(Object o)</i>	This method returns true if the passed element is present within the HashSet
<i>boolean isEmpty()</i>	This method returns true in case the HashSet is empty
<i>Iterator iterator()</i>	This method returns an iterator over the elements present in the HashSet
<i>boolean remove(Object o)</i>	This method helps in removing the specified element from the HashSet if it is present
<i>int size()</i>	This method returns the total number of elements present in the HashSet

```
import java.util.HashSet;
public class HashSetExample {
    public static void main(String args[]) {
        // HashSet declaration
        HashSet<String> hset =
            new HashSet<String>();

        // Adding elements to the HashSet
        hset.add("Apple");
        hset.add("Mango");
        hset.add("Grapes");
        hset.add("Orange");
        hset.add("Fig");
        //Addition of duplicate elements
        hset.add("Apple");
        hset.add("Mango");
        //Addition of null values
        hset.add(null);
        hset.add(null);

        //Displaying HashSet elements
        System.out.println(hset);
    }
}
```

Using Iterator

```
import java.util.HashSet;
import java.util.Iterator;

class IterateHashSet{
    public static void main(String[] args) {
        // Create a HashSet
        HashSet<String> hset = new HashSet<String>();

        //add elements to HashSet
        hset.add("Chaitanya");
        hset.add("Rahul");
        hset.add("Tim");
        hset.add("Rick");
        hset.add("Harry");

        Iterator<String> it = hset.iterator();
        while(it.hasNext()){
            System.out.println(it.next());
        }
    }
}
```

Converting a HashSet to an Array

array=

```
import java.util.HashSet;
class ConvertHashSetToArray{
    public static void main(String[] args) {
        // Create a HashSet
        HashSet<String> hset = new HashSet<String>();

        //add elements to HashSet
        hset.add("Element1");
        hset.add("Element2");
        hset.add("Element3");
        hset.add("Element4");

        // Displaying HashSet elements
        System.out.println("HashSet contains: "+ hset);

        // Creating an Array
        String[] array = new String[hset.size()];
        hset.toArray(array);

        // Displaying Array elements
        System.out.println("Array elements: ");
        for(String temp : array){
            System.out.println(temp);
        }
    }
}
```

Convert HashSet to a List/ArrayList

```
import java.util.HashSet;
import java.util.List;
import java.util.ArrayList;
class ConvertHashSetToArrayList{
    public static void main(String[] args) {
        // Create a HashSet
        HashSet<String> hset = new HashSet<String>();

        //add elements to HashSet
        hset.add("Steve");
        hset.add("Matt");
        hset.add("Govinda");
        hset.add("John");
        hset.add("Tommy");

        // Displaying HashSet elements
        System.out.println("HashSet contains: "+ hset);

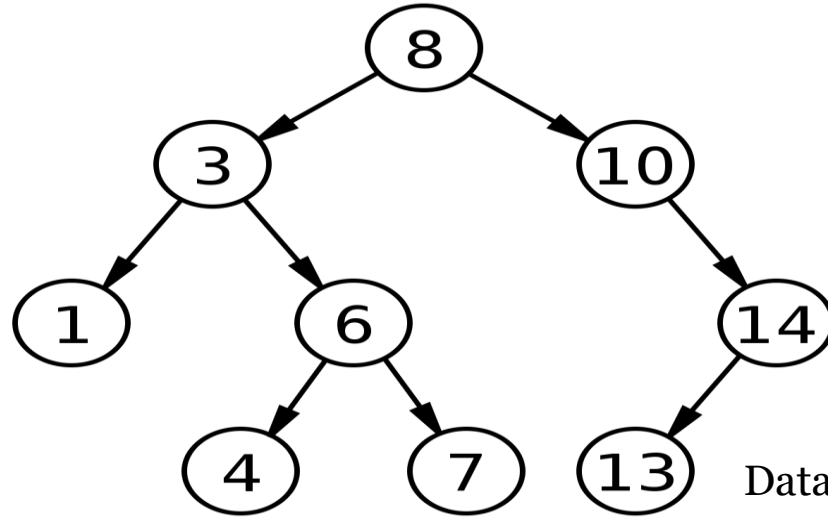
        // Creating a List of HashSet elements
        List<String> list = new ArrayList<String>(hset);

        // Displaying ArrayList elements
        System.out.println("ArrayList contains: "+ list);
    }
}
```

Balanced Tree is the underlying data structure
Duplicates not allowed
Insertion order not preserved
Elements are inserted based on some sorting order
Does not allowed Heterogeneous objects
Null allowed (only once)

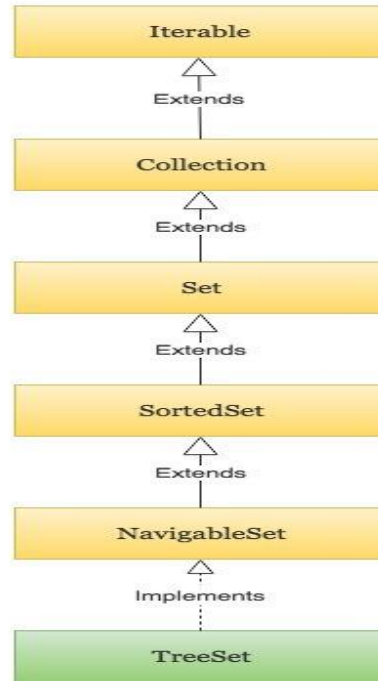
- Java TreeSet class is part of Java's collections framework.
- It implements the NavigableSet interface, which in turn extends the SortedSet interface.
- The elements in a TreeSet are sorted according to their natural ordering.
- provide a custom Comparator to the TreeSet at the time of creation to let it sort the elements based on the supplied comparator

Binary Search Tree (BST)



Data Structure and Algorithm

- The left subtree of a node contains nodes with keys lesser or equal than the node's key.
- The right subtree of a node contains only nodes with keys greater than the node's key.
- The left and right subtree each must also be a binary search tree.



Java TreeSet Class
Hierarchy

```
public class treeset {  
    public static void main(String a[]) {  
  
        TreeSet<Character> t = new TreeSet<>();  
        for(int i='a';i<='z';i++){  
            t.add((char)i);  
        }  
        // t.remove('b');  
        System.out.println(t);  
        System.out.println(t.contains('c'));  
        System.out.println(t.ceiling('b'));  
        // returns the element if present  
        System.out.println(t.floor('b'));  
        System.out.println(t.first());  
        System.out.println(t.last());  
        System.out.println(t.headSet('g')); // g exclusive  
        System.out.println(t.tailSet('g')); // g inclusive  
        System.out.println(t.lower('p')); // strictly lower than p  
    }  
}
```

OUTPUT

[a, c, d, e, f, g, h, i, j, k, l, m, n, o, p, q, r,
s, t, u, v, w, x, y, z]
true

c

a

a

z

[a, c, d, e, f]

[a, c, d, e, f, g]

[a, c, d, e, f, g, h]

o